WoSM Academy

CS102-F26 · DIGITAL DESIGN AND COMPUTER ARCHITECTURE I

Instruction Set Architecture

Lecture 15: the Von Neumann architecture and the DX8's instructions

Dr. Ali Khudiyev

Fall 2026

CC BY-SA 4.0

Today

01 Let’s build a computer

02 Von Neumann architecture

03 The instruction format

04 The DX8 instruction set

05 Summary

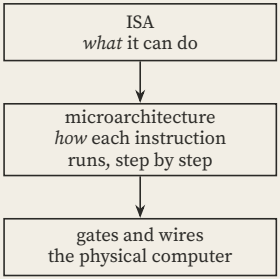
01

Let’s build a computer

From circuits to a computer

- We now have the parts: combinational circuits (Unit 6–9) and memory (Unit 11–14). From here on we build a whole 8-bit computer, the DX8 (Dummy x8).
- Before building anything, we decide what the machine must be able to do.

> **Definition (Instruction set architecture)**
>
> The *ISA* of a computer is the list of its basic operations, the *instructions*, and of how each one is written in binary for the machine.

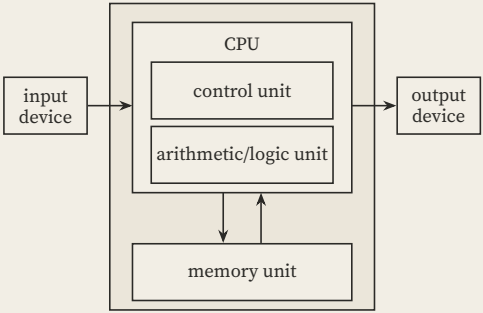


Three levels of design

02

Von Neumann architecture

The Von Neumann architecture

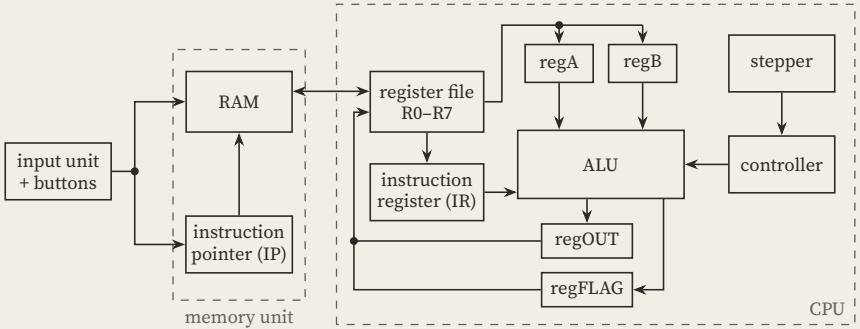


After the book's Figure 6.1

- The CPU: a *control unit* that orchestrates the other parts, and an *ALU* that crunches the numbers.
- One memory that holds the program *and* the data, mixed together.
- Input and output devices, connected to the CPU and memory.

> **Remark**
>
> It names the parts and how they connect, not how to build them or which instructions to offer: that is ours to choose.

The DX8



Simplified from the book’s Figure 6.2; the controller’s control lines are not drawn

The parts of the DX8

CPU

- The register file: eight 8-bit registers, R0 to R7 (Unit 19).
- The ALU: arithmetic ($a + b$, is $a < b$?) and logic (a AND b, NOT a) (Unit 16–18). It reads regA and regB and writes regOUT and regFLAG.
- The controller and the stepper, which run each instruction step by step (Unit 20–21).

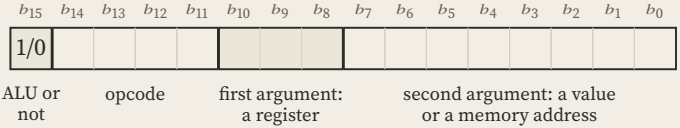
Memory and input

- The RAM: 256 bytes, each read or written by giving its 8-bit address (Unit 19).
- The IP holds the address of the next instruction.
- The input unit: 8 switches for the data, 8 for the address, a *write* button that puts the data at that address, and a *reset* switch.

03

The instruction format

Sixteen bits per instruction



- b_{15}: 1 for an ALU instruction (add, AND, …), 0 for the others (moving data and jumps).
- $b_{14:11}$: the *operation code* or *opcode*, which says which instruction it is.
- $b_{10:8}$: always a register of the register file: 3 bits, R0 to R7.
- $b_{7:0}$, the second byte: an 8-bit value, or an 8-bit memory address, depending on the instruction.

Why 256 bytes

- A memory address fits in the second byte: 8 bits, so $2^8 = 256$ addresses.
- The memory is byte-addressable: each address names a whole byte. (If each address named a single bit, it would be *bit-addressable*.)
- So the DX8 has 256 bytes of RAM, for the program and the data together. An instruction takes 2 of them.

> **An 8-bit computer**
>
> Bytes in memory, operations on byte-long numbers, and 8-bit wide wires between the parts: only 8 bits travel at a time. Hence the name: Dummy x8, or DX8.

04

The DX8 instruction set

Moving data

Instruction	b_{15}	Opcode	Meaning
`load $regX $memAddr`	0	0000	`regX` ← M[`memAddr`]
`store $regX $memAddr`	0	0001	M[`memAddr`] ← `regX`
`move $regX $val`	0	0010	`regX` ← `val`
`arg1 $regX`	0	0011	`regA` ← `regX`
`arg2 $regX`	0	0100	`regB` ← `regX`

From Table 6.2. $M[a]$: the byte at address a of the RAM

- `load` and `store` move a byte between a register and the RAM; `move` puts a value given in the instruction into a register.
- `arg1` and `arg2` copy registers into regA and regB, the two inputs of the ALU.

Computing

Instr.	Code	Meaning
`cmp`	1 0000	`regOUT` ← CMP regA regB
`add`	1 0001	`regOUT` ← `regA` + `regB`
`neg`	1 0010	`regOUT` ← NOT `regA` + 1
`and`	1 0011	`regOUT` ← `regA` AND `regB`
`or`	1 0100	`regOUT` ← `regA` OR `regB`
`xor`	1 0101	`regOUT` ← `regA` XOR `regB`
`not`	1 0110	`regOUT` ← NOT `regA`
`rotl`	1 0111	`regOUT` ← `regA[6:0]_regA[7]`
`rotr`	1 1000	`regOUT` ← `regA[0]_regA[7:1]`
`shiftl`	1 1001	`regOUT` ← `regA[6:0]_0`
`shiftr`	1 1010	`regOUT` ← `0_regA[7:1]`

- The ALU instructions ($b_{15} = 1$) take no arguments: they work on regA and regB and put the result in regOUT.
- `neg` gives $-$`regA` in two's complement (`Unit 16`).
- _ joins bit fields: `rotl` moves every bit one place to the left, and bit 7 comes back in at bit 0.
- `cmp`: the comparator's gt, lt and eq outputs (`Unit 17`) are the flags that the jumps test; the DX8 keeps them in regFLAG.

Jumping

Instruction	b_{15}	Opcode	Meaning
`jmp $val`	0	0101	`regIP` ← `val`
`jmpgt $val`	0	0110	`regIP` ← `val` if the gt flag is set, otherwise `regIP` + 1
`jmplt $val`	0	0111	`regIP` ← `val` if the lt flag is set, otherwise `regIP` + 1
`jmpeq $val`	0	1000	`regIP` ← `val` if the eq flag is set, otherwise `regIP` + 1

- regIP, the instruction pointer, says where the next instruction is. Without jumps, a program runs from one instruction to the next, in order.
- `jmp` always goes to address `val`. The other three go there only if the last `cmp` set their flag: that is how a program makes a decision, or repeats a part of itself.

Writing instructions in binary

Instruction	b_{15}	$b_{14:11}$	$b_{10:8}$	$b_{7:0}$	Hex
`move`: R3 ← 42	0	0010	011	0010 1010	`0x132A`
`store`: M[`0xF0`] ← R5	0	0001	101	1111 0000	`0x0DF0`
`add`, result to R4	1	0001	100	0000 0000	`0x8C00`

- Group the 16 bits in fours and write each group in hexadecimal (Unit 2): `0001 0011 0010 1010` is `0x132A`.
- `add` does not use its second byte; we write those bits as 0. In the book's DX8 circuit, its register field names the register that receives the result (Unit 22).

A tiny program

	Instruction	Meaning	Hex
1	`load`	R2 ← $M[$`0x10`$]$	`0x0210`
2	`load`	R3 ← $M[$`0x11`$]$	`0x0311`
3	`arg1`	`regA` ← R2	`0x1A00`
4	`arg2`	`regB` ← R3	`0x2300`
5	`add`	`regOUT` ← `regA` + `regB`	`0x8C00`

- After these five instructions, regOUT holds $M[$`0x10`$] + M[$`0x11`$]$.
- The program is 10 bytes in the same RAM as the 2 bytes it adds: the Von Neumann idea.
- On the DX8 you type such bytes in with the data and address switches and the write button (`Unit 22`).

Try it

1. b_{15} and the opcode together have 5 bits. How many more instructions would fit?
2. Write $M[\texttt{0x80}] \leftarrow$ R7 (a `store`) in hexadecimal.
3. Which instruction is `0x2806`?
4. `regA` = 1001 0111. What do `rotl`, `shiftr` and `not` give?
5. A program has 100 instructions. How many bytes of RAM are left for its data?

Try it

1. b_{15} and the opcode together have 5 bits. How many more instructions would fit?
   $2^5 = 32$ codes, 20 used: 12 more (5 ALU, 7 others).
2. Write $M[\texttt{0x80}] \leftarrow$ R7 (a `store`) in hexadecimal.
3. Which instruction is `0x2806`?
4. `regA` = 1001 0111. What do `rotl`, `shiftr` and `not` give?
5. A program has 100 instructions. How many bytes of RAM are left for its data?

Try it

1. b_{15} and the opcode together have 5 bits. How many more instructions would fit?
   $2^5 = 32$ codes, 20 used: 12 more (5 ALU, 7 others).
2. Write M[`0x80`] ← R7 (a `store`) in hexadecimal.
   0 0001 111 1000 0000: `0x0F80`.
3. Which instruction is `0x2806`?
4. `regA` = 1001 0111. What do `rotl`, `shiftr` and `not` give?
5. A program has 100 instructions. How many bytes of RAM are left for its data?

Try it

1. b_{15} and the opcode together have 5 bits. How many more instructions would fit?
   $2^5 = 32$ codes, 20 used: 12 more (5 ALU, 7 others).
2. Write M[`0x80`] ← R7 (a `store`) in hexadecimal.
   0 0001 111 1000 0000: `0x0F80`.
3. Which instruction is `0x2806`?
   0 0101 000 0000 0110: `jmp` to address 6.
4. `regA` = 1001 0111. What do `rotl`, `shiftr` and `not` give?

5. A program has 100 instructions. How many bytes of RAM are left for its data?

Try it

1. b_{15} and the opcode together have 5 bits. How many more instructions would fit?
   $2^5 = 32$ codes, 20 used: 12 more (5 ALU, 7 others).
2. Write M[`0x80`] ← R7 (a `store`) in hexadecimal.
   0 0001 111 1000 0000: `0x0F80`.
3. Which instruction is `0x2806`?
   0 0101 000 0000 0110: `jmp` to address 6.
4. `regA` = 1001 0111. What do `rotl`, `shiftr` and `not` give?
   0010 1111, 0100 1011 and 0110 1000.
5. A program has 100 instructions. How many bytes of RAM are left for its data?

Try it

1. b_{15} and the opcode together have 5 bits. How many more instructions would fit?
   $2^5 = 32$ codes, 20 used: 12 more (5 ALU, 7 others).
2. Write $M[\texttt{0x80}] \leftarrow$ R7 (a `store`) in hexadecimal.
   0 0001 111 1000 0000: `0x0F80`.
3. Which instruction is `0x2806`?
   0 0101 000 0000 0110: `jmp` to address 6.
4. `regA` = 1001 0111. What do `rotl`, `shiftr` and `not` give?
   0010 1111, 0100 1011 and 0110 1000.
5. A program has 100 instructions. How many bytes of RAM are left for its data?
   $256 - 2 \cdot 100 = 56$.

05

Summary

Summary

- The ISA lists what a computer can do and how each instruction is written in binary; the microarchitecture and the gates come after it.
- Von Neumann architecture: a CPU (control unit and ALU), one memory for the program and the data, and input and output.
- A DX8 instruction is 16 bits: ALU or not, a 4-bit opcode, a register (R0–R7), and a byte that is a value or an address.
- 8-bit addresses: 256 bytes of byte-addressable RAM.
- 20 instructions: moving data, computing in the ALU (regA and regB to regOUT), and jumps (regIP).

Before the next lecture

- Read the beginning of Chapter 6 and Section 6.1 of the book.
- Write the five instructions that compute $M[\texttt{0x20}]$ AND $M[\texttt{0x21}]$ into regOUT, and give each one in hexadecimal.
- Revise binary addition and numeral systems `(Unit 2)`.

> **Next lecture** `(Unit 16)`
>
> Arithmetic: the half adder, the full adder, multi-bit adders, and two's complement.